



UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

Departamento de Ciencias de la Computación

Carrera de Ingeniería en Software

SecureDataMining DevSecOps

Clasificación de Vulnerabilidades mediante Minería de Datos y
Pipelines DevSecOps

Asignatura: Desarrollo de Software Seguro
Docente: Ing. Geovanny Cudco
Integrantes: Cesar Loor
Gabriel Murillo
Camilo Orrico
Fecha: 18 de junio de 2026

Sangolquí, Ecuador

Índice

1. Introducción	2
2. Ingeniería de Datos y SEMMA	2
2.1. Fase SAMPLE: Consolidación de Datasets	2
2.2. Fase EXPLORE: El desafío del Desbalance	2
2.3. Fase MODIFY: Extracción de Características Avanzada	2
3. Modelado y Evaluación (Fase MODEL/ASSESS)	3
3.1. Justificación Algorítmica	3
3.2. Desempeño del Modelo	3
4. Análisis Estático Heurístico (SAST)	3
5. Ecosistema DevSecOps y Notificaciones	3
6. Conclusiones	4
7. Bibliografía	4

1 Introducción

El paradigma *Shift-Left Security* propone integrar controles de seguridad lo más temprano posible en el ciclo de desarrollo. Este proyecto materializa dicha visión mediante la creación de un ecosistema autónomo que utiliza **Minería de Datos Analítica** para predecir si un cambio en el código fuente (Pull Request) introduce riesgos de seguridad. La solución orquestada en GitHub Actions combina la velocidad del análisis estático con la profundidad del Machine Learning clásico, eliminando la dependencia de servicios externos costosos.

2 Ingeniería de Datos y SEMMA

2.1 Fase SAMPLE: Consolidación de Datasets

Se diseñó un repositorio de datos unificado que ingesta información de fuentes heterogéneas. La diversidad de lenguajes y contextos de vulnerabilidad es clave para la capacidad de generalización del modelo.

- **CodeXGLUE**: Provee fragmentos de código con anotaciones de defectos semánticos.
- **CVEFixes**: Extrae pares de código "vulnerable/corregido" directamente de repositorios vinculados a la base de datos nacional de vulnerabilidades (NVD).
- **D2A/MegaVul**: Aportan cientos de miles de funciones en C/C++ etiquetadas mediante análisis estático industrial.
- **Total de registros limpios**: 389,313.

2.2 Fase EXPLORE: El desafío del Desbalance

Durante la exploración, se determinó que solo el **8.1%** de los fragmentos de código son vulnerables. Esta asimetría es natural en el desarrollo de software (el código seguro es la norma), pero requiere estrategias de entrenamiento específicas como el *cost-sensitive learning* para evitar que el modelo ignore sistemáticamente la clase minoritaria.

2.3 Fase MODIFY: Extracción de Características Avanzada

La transformación de código fuente a vectores numéricos se realizó mediante tres extractores concurrentes:

1. **HashingVectorizer (Truco del Hashing)**: Convierte fragmentos de código en vectores de 262,144 dimensiones (2^{18}). A diferencia de un vocabulario estático, el hashing maneja identificadores de variables y funciones desconocidos sin colisiones significativas y con un consumo de RAM constante.
2. **Análisis AST (Abstract Syntax Tree)**: Utilizando la librería *tree-sitter*, el sistema analiza la gramática del código para extraer:
 - Conteo de operaciones con punteros (crítico para desbordamientos en C++).
 - Profundidad máxima del árbol (indicador de complejidad lógica).
 - Frecuencia de estructuras de control (*if*, *for*, *while*).
3. **Extractor de Taint (Fuentes y Sumideros)**: Identifica mediante regex patrones de flujo de datos inseguros, como entradas de usuario (*scanf*, *argv*) que llegan a funciones peligrosas (*strcpy*, *system*).

3 Modelado y Evaluación (Fase MODEL/ASSESS)

3.1 Justificación Algorítmica

Se seleccionó **XGBoost** (Extreme Gradient Boosting) como clasificador principal debido a su robustez frente a datos tabulares y su capacidad de manejar valores nulos e importancia de variables. Se configuró el parámetro `scale_pos_weight` calculado como:

$$\text{weight} = \frac{\text{count}(\text{safe})}{\text{count}(\text{vulnerable})} \approx 11,4$$

Esto obliga al algoritmo a prestar 11 veces más atención a los errores cometidos sobre la clase vulnerable.

3.2 Desempeño del Modelo

Tabla 1: Métricas detalladas de clasificación

Métrica Global	Valor	Métrica Clase Vulnerable	Valor
Accuracy	92.05 %	Precision	47.69 %
ROC AUC	84.44 %	Recall	30.48 %
CV F1 Mean	0.8122	F1-Score	37.19 %

4 Análisis Estático Heurístico (SAST)

Como red de seguridad adicional, el sistema ejecuta un motor de reglas basado en CWE (*Common Weakness Enumeration*) que garantiza la detección de patrones conocidos independientemente de la probabilidad del modelo ML.

Tabla 2: Reglas de Detección Heurística Implementadas

CWE	Patrón Detectado	Recomendación Técnica
lightgray CWE-120	<code>strcpy, strcat, gets</code>	Usar versiones delimitadas (<code>strncpy</code>).
CWE-78	<code>system, popen, exec</code>	Evitar shell indirectos; usar listas de argumentos.
lightgray CWE-89	SQL Concat / Template Literals	Implementar <i>Prepared Statements</i> / ORM.
CWE-79	<code>innerHTML, document.write</code>	Sanitizar con <code>DOMPurify</code> o usar <code>textContent</code> .
lightgray CWE-134	<code>printf(variable)</code>	Especificar formato estático (<code>printf("%s", var)</code>).
CWE-502	<code>ObjectInputStream.readObject</code>	Migrar a formatos seguros como JSON/Protobuf.

5 Ecosistema DevSecOps y Notificaciones

El pipeline CI/CD actúa como orquestador central, disparando notificaciones ricas en contexto ante fallos:

- GitHub Issues:** Se crea automáticamente un issue etiquetado como `fixing-required` con el desglose de archivos vulnerables, su probabilidad de riesgo y la mitigación específica por CWE.
- Bot de Telegram:** Envía alertas en tiempo real al desarrollador indicando la etapa exacta del pipeline que falló (ML Review, Tests o Deployment).
- Bloqueo de Merge:** El PR queda bloqueado mediante un *Status Check* fallido, impidiendo que el código vulnerable llegue a la rama `test` o `main`.

6 Conclusiones

La solución implementada demuestra un equilibrio entre rigor analítico y operatividad DevSecOps. El uso de la metodología SEMMA permitió pasar de una recolección de datos masiva a un sistema de decisión distribuido que educa al desarrollador en lugar de solo castigar el error. La integración de la ingeniería de características (AST + Hashing) dota al modelo de una visión semántica del código que supera el análisis de texto simple.

7 Bibliografía

Referencias

- [1] OWASP Top 10:2025, “The Most Critical Web Software Security Risks.”
- [2] Chen, T., & Guestrin, C., “XGBoost: A Scalable Tree Boosting System,” 2016.
- [3] SAS Institute, “SEMMA Methodology Overview,” 2026.